

# 唤醒SDK接口设计文档

## 唤醒SDK接口设计文档

版本：1.03

日期：2026-06-22

### 1. 依赖与集成

- AAR 引入
  - Gradle 依赖：

</> Kotlin |

```
1 implementation files('libs/wakeup-release.aar') // 根据提供的实际SDK文件名填写
```

### 2. 核心类与配置

- DuWakeup：SDK单例入口
  - setWakeupCallback(WakeupCallback cb)
  - init(Context context, WakeupInitConfig config)：初始化，config为null时使用默认参数
  - start(DuWakeupIntent intent)：开启内部麦克风采集与唤醒检测，intent为null时使用默认参数
  - feedAudioData(short[] data, int size)：可选，当DuWakeupIntent.isUseFeedData = true 时，由业务侧手动喂入 16k/mono/16-bit PCM；size 为本次有效采样点数
  - stop()、release()、getVersion()、getStatus()、setLogLevel(int)、getCurrentWakeupWords()
- WakeupInitConfig：初始化配置
  - isCustomWords：是否使用自定义唤醒词（默认为false）
  - wakeWordModel.modelPath：模型文件绝对路径（默认使用SDK内置资源，也可指定外部模型文件地址）
  - wakeConfig：唤醒行为配置

- `wakeInterval` : 最大唤醒时间窗口 (ms), 窗口内最多触发一次唤醒, 默认 1900ms
- `wakeThreshold` : 唤醒阈值 [0.0, 1.0]
- `saveWakeupAudio` : 是否保存唤醒音频 (true/false)
- `wakeupAudioPath` : 保存路径
- `detectionWindowsFrames` : 检测窗口帧数 (默认 40)
- `thresholdFramesCount` : 达到阈值的连续帧数 (默认 1)
- `customWakeThreshold` : 自定义唤醒词阈值 (范围: 0.0-1.0, 默认0.1), 仅在自定义唤醒词模式下使用
- `DuWakeupIntent` : 唤醒意图配置
  - `audioSource` : 默认 `MediaRecorder.AudioSource.VOICE_RECOGNITION`, 使用内部录音。
  - `isEnableAEC` : 是否使用AEC, 只支持内置录音机 (默认false)
  - `isEnableNS` : 是否使用NS, 只支持内置录音机 (默认false)
  - `isUseFeedData` : 是否手动传入音频 (默认false, sdk内自动录音)
  - `wakeupWords` : 唤醒词列表 (只有自定义唤醒模式需要, 模型内置默认唤醒词“灵犀灵犀”)
- `WakeupEventInfo`
  - `word` : 唤醒词
  - `confidence` : 置信度 (分数, 0~1)
  - `time` : 发生时间戳 (ms)
  - `audioData` : 唤醒时刻的前2s音频数据, 可用于保存排查问题
  - `sn` : 唤醒事件唯一标识
- `DuWakeupError`
  - `code` : 错误码
    - -100: 模型初始化失败 (模型文件不存在)
    - -200: 未初始化就调用start
    - -201: 启动录音失败或录音数据错误
    - -202: 启动唤醒失败
    - -300: 未初始化就调用stop
  - `message` : 错误消息
- `WakeupStatus`
  - UN\_INIT: 未初始化
  - INIT: 已初始化
  - START: 运行中
  - STOP: 已停止

---

### 3. 回调接口

源码签名：

```
</> Java |
1 public interface WakeupCallback {
2     void onInit(); // 初始化完成
3     void onStart(); // 启动监听成功
4     void onWakeup(WakeupEventInfo info); // 触发唤醒: info.confidence
    为唤醒分数
5     void onAudioData(short[] audioData); // 连续音频数据: 16k、mono、
    16bit、LE
6     void onWakeupFrameThreshold(float var1); // 模型推理分数, 用作调试
7     void onStop(); // 停止监听成功
8     void onError(DuWakeupError error); // 异常回调
9     void onRelease(); // 资源释放完成
10 }
```

建议：

- UI 波形直接使用 `onAudioData` 的连续数据绘制（例如每 20ms 计算一次能量点，1 秒约 50 个点，平滑显示）。
- 唤醒分数展示使用 `onWakeup(info.confidence)`。

## 4. 典型用法

```
</> Java |
1 // 设置日志等级（可选）
2 DuWakeup.getInstance().setLogLevel(Log.INFO);
3
4 // 设置回调
5 DuWakeup.getInstance().setWakeupCallback(new WakeupCallback() {
6     @Override public void onInit() {}
7     @Override public void onStart() {}
8     @Override public void onWakeup(WakeupEventInfo info) {
9         // info.confidence 为唤醒分数 (0~1)
10    }
11    @Override public void onAudioData(short[] audioData) {
12        // 计算能量并更新波形
13        // 例: energy = mean(x^2); dbNorm = (10*log10(1+energy))/100, 限制到
        [0,1]
14    }
15    @Override public void onStop() {}
16    @Override public void onError(DuWakeupError error) {}
}
```

```

17  @Override public void onRelease() {}
18  });
19
20  // 初始化 (建议使用 filesDir 下的模型路径)
21  WakeupInitConfig cfg = new WakeupInitConfig();
22  cfg.isCustomWords = false; // 是否使用自定义唤醒词, 按需设置
23  cfg.wakeConfig.wakeThreshold = 0.90f; // 固定唤醒阈值, 需根据实际情况微调
24  cfg.wakeConfig.customWakeThreshold = 0.10f; // 自定义唤醒阈值, 需根据实际情况
    微调
25  DuWakeup.getInstance().init(cfg, null);
26
27  // 启动/停止监听 (使用内部收音)
28  DuWakeupIntent intent = new DuWakeupIntent(); // 可保留默认 audioSource
29  intent.wakeupWords = new String[]{"小度小度"}; // 设置自定义唤醒词
30  DuWakeup.getInstance().start(intent);
31  // 查询当前唤醒词
32  String[] words = DuWakeup.getInstance().getCurrentWakeupWords();
33  if (words != null) {
34      // 使用唤醒词列表
35      for (String word : words) {
36          Log.d("Wakeup", "唤醒词: " + word);
37      }
38  }
39  // ...
40  DuWakeup.getInstance().stop();
41
42  // 退出时释放
43  DuWakeup.getInstance().release();

```

可选：手动喂入音频（仅在不使用内部麦克风时）

</>

Java

```

1  short[] pcm16leMono16k = ...;
2  int validSize = ...; // 有效采样点数, 可等于 pcm16leMono16k.length
3  DuWakeup.getInstance().feedAudioData(pcm16leMono16k, validSize);

```

## 5. 注意事项与最佳实践

- 权限：需要 `RECORD_AUDIO`。如需将音频保存到外部目录，请根据系统版本处理存储权限或使用 SAF。
- 音频要求：PCM、单声道、16k 采样、16 位、小端序。
- 回调中避免耗时操作，建议在工作线程处理，再回主线程更新 UI。

- 接口调用顺序： `setWakeupCallback` → `init` → `start` → `stop` → `release` 。
- 

## 6. 常见问题

- 待补充
- 

## 7. 版本与兼容性

- 支持架构：AAR 内置常见 ABI（arm64-v8a、armeabi-v7a、x86、x86\_64）。
- 最低系统版本：建议 Android 7.0（API 24）及以上。